

Paradigma OO – TypeScript + Smalltalk

Paradigma Orientado a Objetos

TypeScript · Smalltalk · Los 4 pilares

- Paradigmas y Lenguajes de Programación 2026
- UNTDF – Instituto IDEI | Módulo IV – Semana 7

¿Qué sumamos hoy?

¿Qué sumamos hoy al cursado?

Paradigmas vistos en el cursado:

- Funcional → ¿qué transformar? — funciones puras, inmutabilidad
- Lógico → ¿qué es verdad? — hechos, reglas, motor de inferencia
- OO → (hoy) — extensión del imperativo que encapsula estado y comportamiento

El imperativo como base común:

OO **extiende** el imperativo — agrega encapsulamiento, herencia y polimorfismo sobre la base de instrucciones secuenciales y estado mutable.

¿Qué tiene OO que los paradigmas funcional y lógico no tienen?

Estado encapsulado en objetos, comunicación por mensajes, identidad de entidades

Los 4 pilares — adelanto

Los 4 pilares del paradigma OO

Estos cuatro conceptos son el contenido central de la clase y del **Parcial 1**.

Los trabajaremos primero en **Smalltalk** (OO puro) y luego en **TypeScript** (OO pragmático).

Pilar	Pregunta que responde	Ejemplo en una línea
Encapsulamiento	¿Cómo protejo el estado interno?	private saldo: number
Abstracción	¿Qué expongo al exterior?	abstract area(): number
Herencia	¿Cómo reutilizo y especializo?	class Perro extends Animal
Polimorfismo	¿Cómo trato distintos tipos uniformemente?	formas.forEach(f => f.area())

Simula 67 — el primer paso

Origen: Simula 67

Nygaard & Dahl — Noruega, 1967

Primer lenguaje con objetos y clases — diseñado para **simular sistemas físicos**.

- Un barco tiene estado (velocidad, posición) y comportamiento (moverse, frenar)
- Múltiples barcos son instancias de la misma clase Barco
- No era "OO puro" — era ALGOL extendido con mecanismos de simulación

Innovación central:

Agrupar **datos** (estado) con las **operaciones** que los modifican → encapsulamiento

Antes de Simula: los datos eran estructuras separadas de las funciones que las procesaban.

Simula dijo: *el barco sabe cómo moverse — no me lo decís vos desde afuera.*

Alan Kay — la visión original

Alan Kay — Xerox PARC, 1970s

La cita fundacional:

"The big idea is 'messaging' — that is what the kernel of Smalltalk/Squeak is all about. The key in making great and growable systems is much more to design how its modules communicate rather than what their internal properties and behaviors should be."

— Alan Kay, 1998

Los tres principios originales de Smalltalk:

1. Todo es un objeto — incluyendo números, booleanos, clases y métodos
 2. Los objetos se comunican únicamente por mensajes
 3. Cada objeto tiene su propia memoria — encapsulamiento absoluto
- Dato: Kay consideraba que el nombre "OO" fue un error — el centro no son los objetos, son los mensajes.

Kay vs. lo que el mundo hizo después

La tensión central del paradigma OO

Kay quería esto:

Objetos autónomos como células biológicas — cada uno con su propia lógica interna, comunicándose **solo por mensajes**. Sin clases rígidas si no eran necesarias. Sin primitivos. Todo objeto.

El mundo hizo esto:

Java, C++, Python, TypeScript — lenguajes con **clases** como mecanismo central, primitivos por performance, verificación estática para escalar en equipo.

La pregunta:

¿Importa la pureza filosófica o el resultado práctico?

No hay una respuesta única. La tensión **pureza vs. pragmatismo** es la pregunta de diseño de lenguajes más importante del paradigma OO — y es lo que esta clase explora.

Evolución de los lenguajes OO

- Tendencia: cada generación gana en pragmatismo, pierde en pureza.
- Pregunta clave de la clase: ¿importa la pureza, o solo el resultado?

Lenguaje	Año	Característica OO
Simula 67	1967	Clases, objetos, herencia
Smalltalk-80	1980	Todo es objeto, mensajes, metaclasses — OO puro
C++	1983	OO sobre C — herencia múltiple, sin GC, tipos estáticos
Eiffel	1986	OO con contratos formales (precondiciones, postcondiciones)
Java	1995	OO con JVM, GC, herencia simple, interfaces — OO pragmático
Python	1991	OO multi-paradigma, duck typing dinámico
JavaScript	1995	OO prototípico (herencia sin clases hasta ES6)
TypeScript	2012	OO sobre JS — tipado estático estructural, class ES6+

"Todo es un objeto" — sin excepciones

Smalltalk — "Todo es un objeto"

No hay primitivos. Ninguno.

En Smalltalk cada valor es una instancia de una clase:

- 5 → instancia de SmallInteger → hereda de Integer → Number → Magnitude → Object
- true → instancia de True → hereda de Boolean → Object
- \$A → instancia de Character → hereda de Magnitude → Object
- 'hola' → instancia de String → hereda de Collection → Object
- La clase Animal → instancia de su metaclasses Animal class

En TypeScript — primitivos existen:

- Autoboxing = TypeScript/JS envuelven primitivos en objetos temporalmente. Smalltalk no necesita simularlo — son objetos de verdad.

"Todo es un objeto" — en código

```
5 class.           "→ SmallInteger"
5 class superclass. "→ Integer"
5 class superclass superclass. "→ Number"
5 factorial.       "→ 120"
5 between: 1 and: 10. "→ true"
5 printString.    "→ '5'"

true class.       "→ True"
false class.      "→ False"
true & false.     "→ false"
true | false.     "→ true"
(3 > 2) ifTrue: ['mayor'] ifFalse: ['menor']. "→ 'mayor'"

SmallInteger superclass. "→ Integer"
SmallInteger allSubclasses. "→ todas las subclases"
SmallInteger methodDictionary. "→ todos los métodos"
SmallInteger canUnderstand: #factorial. "→ true"
```

Cascades y `yourself` – sintaxis expresiva

```
| col |  
col := OrderedCollection new.  
col add: 'primero'.  
col add: 'segundo'.  
col add: 'tercero'.  
  
| col |  
col := (OrderedCollection new)  
  add: 'primero';  
  add: 'segundo';  
  add: 'tercero';  
  yourself.    "← retorna la colección, no el último add:"  
  
// En TypeScript con fluent interface / builder pattern:  
class ColeccionBuilder {  
  private items: string[] = [];  
  add(item: string): this { this.items.push(item); return this; }  
  build(): string[] { return this.items; }  
}  
  
const col = new ColeccionBuilder()  
  .add("primero")  
  .add("segundo")  
  .add("tercero")  
  .build();
```

Mensajes — los tres tipos

Mensajes — tres tipos, una filosofía

Mensajes unarios — sin argumentos (mayor precedencia)

Mensajes binarios — un argumento, notación infija

Mensajes de palabra clave — uno o más argumentos (menor precedencia)

- Regla: unarios > binarios > palabras clave. Usá paréntesis para cambiar el orden.

Clases y herencia en Smalltalk

```
Object subclass: #Animal
  instanceVariableNames: 'nombre'
  category: 'Ejemplo'.

Animal subclass: #Perro
  instanceVariableNames: 'raza'
  category: 'Ejemplo'.

"Accessors"
Animal >> nombre      ^ nombre
Animal >> nombre: n    nombre := n
Perro  >> raza: r      raza := r

"Comportamiento"
Animal >> hablar      ^ 'Soy un animal'
Perro  >> hablar      ^ 'Guau! Soy ', nombre, ', un ', raza

| animales |
animales := OrderedCollection new.
animales add: (Animal new nombre: 'Genérico').
animales add: (Perro new nombre: 'Rex'; raza: 'Labrador'; yourself).

animales do: [:a | Transcript showCr: a hablar].
"- Soy un animal"
"- Guau! Soy Rex, un Labrador"
```

Bloques — las lambdas de Smalltalk

"Bloque con argumentos"

```
| cuadrado |  
cuadrado := [:x | x * x].  
cuadrado value: 5.    "→ 25"  
cuadrado value: 12.   "→ 144"
```

"Bloque como condicional — ifTrue: recibe un bloque"

```
(temperatura > 30) ifTrue: ['Calor'] ifFalse: ['Fresco'].
```

"Iteración — do: recibe un bloque"

```
 #(1 2 3 4 5) do: [:n | Transcript showCr: n printString].
```

"Collect — equivalente a map"

```
 #(1 2 3) collect: [:n | n * 2].    "→ #(2 4 6)"
```

"Select — equivalente a filter"

```
 #(1 2 3 4 5) select: [:n | n > 3].    "→ #(4 5)"
```

```
const cuadrado = (x: number) => x * x;  
cuadrado(5); // 25
```

```
[1, 2, 3, 4, 5].forEach(n => console.log(n));  
[1, 2, 3].map(n => n * 2);           // [2, 4, 6]  
[1, 2, 3, 4, 5].filter(n => n > 3); // [4, 5]
```

Metaclasses — las clases son objetos

Metaclasses — lo que hace único a Smalltalk

La cadena de metaclasses

Métodos de clase via metaclass

Comparación con TypeScript

- Diferencia: en TypeScript los métodos static son un atajo sintáctico. Las clases no son ciudadanos de primera clase — no podés inspeccionar su metaclass ni enviarles mensajes arbitrarios en runtime.

Smalltalk hoy – Pharo

Pharo – Smalltalk moderno (2026)

Ejecutar Smalltalk hoy, sin instalar nada

¿Para qué sirve aprender Pharo hoy?

El IDE de Pharo – todo es un objeto, incluso el IDE

El IDE de Pharo está escrito en Pharo. Podés inspeccionar el IDE con el IDE. No hay separación entre herramienta y lenguaje.

- Recurso: <https://pharo.org/> → IDE descargable en 2 minutos.

```
"En Pharo Playground –  
ejecutá esto:"  
| col |  
col :=  
OrderedCollection new.  
1 to: 10 do: [:i | col  
add: i * i].  
col select: [:n | n >  
20].  
"→ OrderedCollection  
(25 36 49 64 81 100 )"
```

Clases en TypeScript – encapsulamiento

```
class CuentaBancaria {  
  private saldo: number;      // solo accesible internamente  
  protected titular: string;  // accesible en subclases  
  readonly numero: string;     // inmutable post-construcción  
  
  constructor(titular: string, numero: string, saldoInicial: number) {  
    this.titular = titular;  
    this.numero = numero;  
    this.saldo = saldoInicial;  
  }  
  
  depositar(monto: number): void {  
    if (monto > 0) this.saldo += monto;  
  }  
  
  retirar(monto: number): boolean {  
    if (monto > 0 && monto <= this.saldo) {  
      this.saldo -= monto;  
      return true;  
    }  
    return false;  
  }  
  
  obtenerSaldo(): number { return this.saldo; }  
}  
  
const c = new CuentaBancaria("Ana", "001-99", 1000);  
c.depositar(500);  
console.log(c.obtenerSaldo()); // 1500  
// c.saldo = 9999;              // ❌ Error de compilación
```


Shorthand de constructor — menos boilerplate

Shorthand constructor — TypeScript idiomático

Sin shorthand (verboso):

Con shorthand (idiomático):

Mixto — con valores por defecto:

- Regla: si el constructor solo asigna propiedades → shorthand. Si tiene lógica (validaciones, transformaciones) → forma explícita.

```
class Punto {  
  private x: number;  
  private y: number;  
  constructor(x: number, y: number) {  
    this.x = x;  
    this.y = y;  
  }  
}  
  
class Punto {  
  constructor(private x: number, private y: number) {}  
  
  distanciaAlOrigen(): number {  
    return Math.sqrt(this.x ** 2 + this.y ** 2);  
  }  
}  
  
class Configuracion {  
  constructor(  
    public readonly host: string,  
    public puerto: number = 3000,  
    private debug: boolean = false  
  ) {}  
}  
  
const cfg = new Configuracion("localhost");  
console.log(cfg.host, cfg.puerto); // "localhost" 3000
```

Herencia con `extends` — polimorfismo en acción

```
class Animal {  
  constructor(protected nombre: string) {}  
  hablar(): string { return `Soy ${this.nombre}, un animal.`; }  
}  
  
class Perro extends Animal {  
  constructor(nombre: string, private raza: string) {  
    super(nombre); // OBLIGATORIO - llama al constructor del padre  
  }  
  hablar(): string { return `${this.nombre} dice: ¡Guau!`; }  
  describirse(): string { return `${this.nombre} es un ${this.raza}`; }  
}  
  
class PerroEntrenado extends Perro {  
  hablar(): string { return `${super.hablar()} (bien entrenado)`; }  
}  
  
const animales: Animal[] = [  
  new Animal("Genérico"),  
  new Perro("Rex", "Labrador"),  
  new PerroEntrenado("Max", "Poodle"),  
];  
  
animales.forEach(a => console.log(a.hablar()));  
// "Soy Genérico, un animal."  
// "Rex dice: ¡Guau!"  
// "Max dice: ¡Guau! (bien entrenado)"
```

Interfaces — contratos estructurales

```
interface Describable { describirse(): string; }
interface Serializable { serializar(): string; }

class Perro extends Animal implements Describable, Serializable {
  constructor(nombre: string, private raza: string) { super(nombre); }

  hablar(): string { return `${this.nombre}: ¡Guau!`; }

  describirse(): string { return `${this.nombre} es un ${this.raza}`; }

  serializar(): string {
    return JSON.stringify({ nombre: this.nombre, raza: this.raza });
  }
}

function mostrar(obj: Describable): void {
  console.log(obj.describirse());
}

mostrar(new Perro("Rex", "Labrador")); // ✓
```

Tipado estructural – duck typing estático

Tipado estructural vs. Nominal vs. Dinámico

Duck typing estático – ejemplo

- Gabrielli & Martini (2023): TypeScript es un lenguaje gradualmente tipado – la compatibilidad se define por estructura, no por nombre de clase.

Aspecto	TypeScript (estructural)	Java (nominal)	Smalltalk (dinámico)
Base de compatibilidad	Forma del objeto	Declaración explícita	Responde al mensaje
implements	Opcional	Obligatorio	No existe
Verificación	Compilación	Compilación	Runtime
Flexibilidad	Alta	Media	Máxima
Seguridad	Alta (estática)	Alta (estática)	Solo en runtime

Modificadores de acceso y clases abstractas

Modificadores y abstracción

Clase abstracta en TypeScript

Modificador	Acceso	Smalltalk equivalente
public	Desde cualquier lugar	Mensajes (todo)
protected	Clase + subclases	No existe formalmente
private	Solo la clase	Instancia vars siempre privadas
readonly	Immutable post-init	—
abstract (clase)	No instanciable	Convención — no existe keyword
abstract (método)	Obliga implementación en subclase	^ self subclassResponsibility

Composición vs. herencia

Composición vs. herencia — elegir bien

El anti-patrón: herencia por reutilización

La solución: composición

- Regla: usá herencia cuando la subclase es-un tipo de la superclase (relación semántica real). Usá composición cuando solo querés reutilizar comportamiento.

```
// ✗ Pila NO ES UN Array - herencia por conveniencia
class Pila extends Array<number> {
  tope(): number { return this[this.length - 1]; }
}

const p = new Pila();
p[0] = 99; // ✓ TypeScript lo permite - pero rompe la
           // abstracción de Pila
p.splice(0, 1); // ✓ TypeScript lo permite - viola el contrato
               // de una Pila

// ✓ Pila TIENE UN array - no es un array
class Pila<T> {
  private items: T[] = [];

  push(item: T): void { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  tope(): T | undefined { return this.items[this.items.length -
1]; }
  estaVacia(): boolean { return this.items.length === 0; }
}
```

El dominio — formas geométricas

Dominio compartido — mismo problema, dos lenguajes

El problema:

- Jerarquía de formas geométricas (Círculo, Rectángulo, Triángulo).
- Cada forma tiene un color y puede calcular su área y perímetro.
- Una colección itera todas las formas polimórficamente.
- El cliente no sabe (ni necesita saber) el tipo concreto de cada forma.

¿Por qué este dominio?

- Pequeño pero estructuralmente completo — usa los 4 pilares.
- Sebesta (2019) lo usa para contrastar OO puro vs. pragmático.
- Revela cómo cada lenguaje fuerza (o no) la abstracción.

Formas en Smalltalk – OO puro

```
Object subclass: #Forma
  instanceVariableNames: 'color'
  category: 'Geometria'.

Forma subclass: #Circulo
  instanceVariableNames: 'radio'.
Forma subclass: #Rectangulo
  instanceVariableNames: 'ancho alto'.

Forma >> area
  ^ self subclassResponsibility "error en runtime si subclase no implementa"

Circulo >> radio: r    radio := r
Circulo >> area        ^ Float pi * radio * radio
Circulo >> perimetro    ^ 2 * Float pi * radio

Rectangulo >> ancho: a alto: h    ancho := a. alto := h.
Rectangulo >> area                ^ ancho * alto
Rectangulo >> perimetro            ^ 2 * (ancho + alto)

| formas |
formas := OrderedCollection new.
formas add: (Circulo new radio: 5; color: 'rojo'; yourself).
formas add: (Rectangulo new ancho: 4 alto: 6; color: 'azul'; yourself).

formas do: [:f | Transcript showCr: f area printString].
"→ 78.53981..."
"→ 24"
```


Formas en TypeScript – definición base

```
abstract class Forma {
  constructor(protected color: string) {}
  abstract area(): number;
  abstract perimetro(): number;

  toString(): string {
    return `${this.constructor.name}(${this.color}) → área=${this.area().toFixed(2)}, p=${this.perimetro().toFixed(2)}`;
  }
}

class Circulo extends Forma {
  constructor(private radio: number, color: string) { super(color); }
  area(): number { return Math.PI * this.radio ** 2; }
  perimetro(): number { return 2 * Math.PI * this.radio; }
}

class Rectangulo extends Forma {
  constructor(private ancho: number, private alto: number, color: string) {
    super(color);
  }
  area(): number { return this.ancho * this.alto; }
  perimetro(): number { return 2 * (this.ancho + this.alto); }
}

const formas: Forma[] = [
  new Circulo(5, "rojo"),
  new Rectangulo(4, 6, "azul"),
];
formas.forEach(f => console.log(f.toString()));
```

Extender el dominio — Principio Open/Closed

```
class Triangulo extends Forma {
  constructor(
    private base: number,
    private lado1: number,
    private lado2: number,
    color: string
  ) { super(color); }

  area(): number {
    const s = (this.base + this.lado1 + this.lado2) / 2;
    return Math.sqrt(s * (s - this.base) * (s - this.lado1) * (s - this.lado2));
  }

  perimetro(): number { return this.base + this.lado1 + this.lado2; }
}

// El cliente polimórfico no cambió ni una línea:
const formas: Forma[] = [
  new Circulo(5, "rojo"),
  new Rectangulo(4, 6, "azul"),
  new Triangulo(3, 4, 5, "verde"), // ← solo esto se agregó
];

formas.forEach(f => console.log(f.toString()));
// "Circulo(rojo) → área=78.54, p=31.42"
// "Rectangulo(azul) → área=24.00, p=20.00"
// "Triangulo(verde) → área=6.00, p=12.00"
```

Comparación directa — tabla

Smalltalk vs. TypeScript — mismo dominio

Aspecto	Smalltalk	TypeScript
Método abstracto	<code>^ self subclassResponsibility</code>	<code>abstract area(): number</code>
Error si no se implementa	Runtime (al llamar)	Compilación (inmediato)
Iteración	Mensaje do: sobre Collection	Método .forEach() sobre Array
Instanciación	<code>Circulo new radio: 5; yourself</code>	<code>new Circulo(5, "rojo")</code>
Tipo de variable	Dinámico — cualquier objeto	Estático — <code>Forma[]</code>
Error de tipo	Runtime	Compilación

Reflexión guiada

Comparación — análisis del dominio

Tres preguntas para discutir:

1. ¿En qué caso Smalltalk es más expresivo?

- Mensajes como ciudadanos de primera clase
- Cascades — construir objetos en una línea
- Metaclases — las clases mismas son configurables en runtime

2. ¿En qué caso TypeScript es más robusto para equipos?

- Errores en compilación → el IDE te avisa antes de ejecutar
- abstract obliga a la implementación antes de deployar
- Tipado estructural permite verificar contratos en el repositorio

3. ¿Cuál sigue más la visión original de Alan Kay?

- Smalltalk — sin duda
- TypeScript hace concesiones al desarrollo industrial moderno
- La pregunta interesante: ¿Kay estaría de acuerdo con esas concesiones?

OO + LLMs — el paradigma dominante

OO en el mundo moderno

El paradigma más entrenado

Los LLMs (GPT, Copilot, Claude) fueron entrenados con enormes repositorios OO — Python, Java y TypeScript dominan el corpus.

- Implicación: cuando el prompt pide una jerarquía con abstract, implements y polimorfismo, Copilot suele completar mejor.

```
// Prompt a Copilot:  
// "Modelá Email, SMS y Push con una interfaz común"  
interface Notificacion { enviar(): void }  
  
abstract class Canal implements Notificacion {  
    constructor(protected destino: string) {}  
    abstract enviar(): void;  
}
```

Patrones GoF – OO destilado

Patrones de Diseño GoF (Gang of Four, 1994)

Los patrones GoF son **soluciones OO reutilizables a problemas recurrentes**.

- Template Method ya lo usamos sin nombrarlo: toString() en Forma llama a area() — definido en la clase abstracta, implementado en subclases.

Patrón	Tipo	Qué resuelve	Ejemplo
Strategy	Comportamiento	Algoritmos intercambiables sin modificar el cliente	Ordenamiento — burbuja, merge
Observer	Comportamiento	Notificar a múltiples objetos cuando cambia el estado	EventEmitter, Redux
Factory Method	Creación	Delegar la instanciación a subclases	crearForma(tipo): Forma
Decorator	Estructura	Agregar responsabilidades dinámicamente	Middleware en Express.js

Síntesis – Smalltalk vs. TypeScript

Smalltalk realiza la visión de Kay con más pureza. TypeScript hace concesiones al desarrollo industrial moderno.

Concepto OO	Smalltalk	TypeScript
Todo es objeto	✓ absoluto	✗ primitivos (number, boolean)
Mensajes	✓ única forma de interacción	✓ llamadas a métodos
Clases como objetos	✓ metaclasses	⚠ parcial — métodos static
Herencia	✓ simple (subclass:)	✓ simple (extends)
Polimorfismo	✓ dinámico puro	✓ dinámico + verificación estática
Encapsulamiento	✓ inst. vars siempre privadas	✓ private/protected/readonly
Clases abstractas	⚠ subclassResponsibility (runtime)	✓ abstract (compilación)
Interfaces	✗ no existen	✓ duck typing estático
Bloques/lambdas	✓ objetos de primera clase	✓ arrow functions
Operadores redefinibles	✓ son mensajes	✗ operadores fijos
Verificación de tipos	Runtime	Compilación

Los 4 pilares – síntesis con código

Los 4 pilares en un solo fragmento

Pilar	Definición en una línea	TypeScript	Smalltalk
Encapsulamiento	Ocultar el estado interno	<code>private saldo: number</code>	Variables de inst. siempre privadas
Abstracción	Exponer solo la interfaz	<code>abstract area(): number</code>	<code>^ self subclassResponsibility</code>
Herencia	Reutilizar y especializar	<code>class Perro extends Animal</code>	<code>Animal subclass: #Perro</code>
Polimorfismo	Un contrato, muchas formas	<code>formas.forEach(f => f.area())</code>	<code>`formas do: [:f area]`</code>

Mapa de paradigmas del cursado

Los tres paradigmas del cursado – mapa comparativo

- OO es una extensión del imperativo – por eso no se estudia como paradigma separado sino como evolución.
- Para el Parcial 1: deberías poder contrastar al menos dos de estos paradigmas para el mismo problema y argumentar las decisiones de diseño.

Conceptos clave — Parcial 1

Los 4 pilares — definición + ejemplo en código:

1. Encapsulamiento → private, protected, accessors.
2. Abstracción → abstract, interfaces.
3. Herencia → extends, super().
4. Polimorfismo → override, despacho dinámico.

Diferencias clave a argumentar:

- OO puro (Smalltalk) vs. OO pragmático (TypeScript) — todo es objeto vs. primitivos.
- Polimorfismo por herencia (extends) vs. por interfaz (implements).
- Tipado nominal (Java) vs. estructural (TS) vs. duck typing dinámico (Smalltalk).
- Composición vs. herencia — cuándo elegir cada una.